

members. And we have another class B, and we want this class B to be inherited from class A. The following code shows how to establish the Parent-Child relationship between Class A and Class B.

```
class A
{
    //Members
}
class B : A ← Establishing Parent/Child Relationship
{
    //Consuming the Members of A from here
}
```

So, this is the basic process for establishing a Parent/Child relationship in C#. Now, let us see the basic syntax to establish a Parent/Child relationship between classes. The syntax is given below.

[<modifiers>] class <child class> : <parent class>

Here, the terminology Parent Class and Child Class can also be called Base Class (Superclass) and Derived Class (Subclass). So, in our example,

A => Parent/ Base/ Superclass (all meaning the same; you can use any term)

B => Child/ Derived/ Subclass (all meaning the same; you can use any term)

Note: In Inheritance, the Child class can consume members of its Parent class as if it is the owner of those members (except private members of the parent).

Why Child Cannot Consume Private Members of Parent?

Generally, Children have rights to their Parent's Property. As a Child, tomorrow, I can take over my father's business. I can take over my Father's Properties (Car, Buildings, Money, whatever it is). But I cannot take over my father's job. The reason is the Job whatever my father is doing may be based on his qualifications and his experiences. And tomorrow I cannot take over his particular job. So, the Job is completely private to my father. And that is not inherited by me. But remain everything, business, money, properties, whatever I will take. Take over everything except the private members.

The same principle is also applied to Inheritance. So, the child class will consume all the members of the Parent class except the private members.

Example to Understand Inheritance in C#:

Let us see a simple example to understand Inheritance in C#. Let us create a class with two methods, as shown below.

```
class A
{
    public void Method1()
    {
        Console.WriteLine("Method 1");
    }
    public void Method2()
    {
    }
}
```

```

    {
        Console.WriteLine("Method 2");
    }
}

```

Here, we have created class A with two public methods, i.e., Method1 and Method2. Now, I want the same two methods in another class, i.e., class B. One way to do this is to copy the above two methods and paste them into class B as follows.

```

class B
{
    public void Method1()
    {
        Console.WriteLine("Method 1");
    }
    public void Method2()
    {
        Console.WriteLine("Method 2");
    }
}

```

If we do this, then it is not code re-usability. It is code rewriting that affects the size of the application. So, without rewriting, what we need to do is, we need to perform inheritance here as follows. Here, class B is inherited from class A, and hence, inside the Main method, we create an instance of class B and invoke the methods which are defined in Class A.

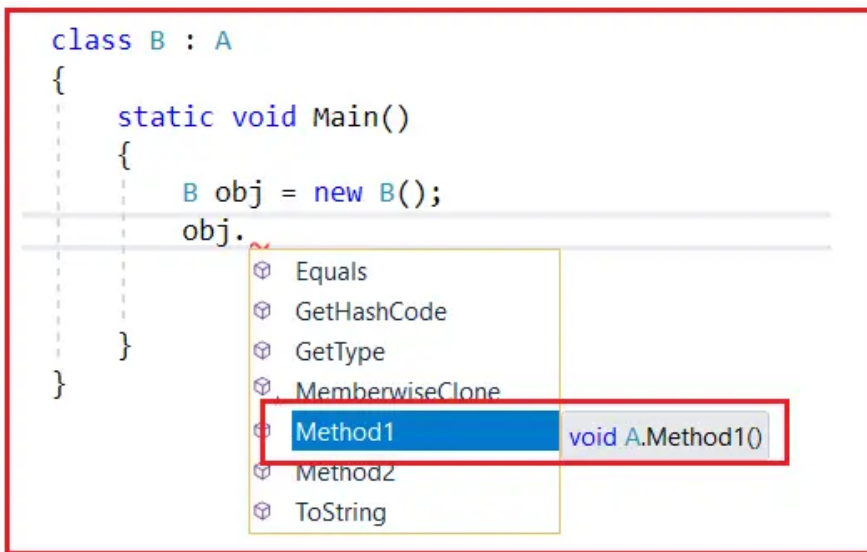
```

class B : A
{
    static void Main()
    {
        B obj = new B();
        obj.Method1();
        obj.Method2();
    }
}

```

Once you perform the Inheritance, class B can take the two members defined in class A. Why? Because all the properties of a Parent belong to Children. Here, class A is the Parent/Super/Base class, and Class B is the Child/Sub/Derived class.

Let us understand one more thing. Please observe the following image when we say obj. you can see the intelligence it is showing the two Methods, i.e., Method1 and Method2. So, the child class can consume the members of the parent class as if it is the owner. Now, if you see the description of either Method1 or Method2, it is showing void A.Method1() and void A.Method2(). That means Method1 or Method2 belongs to class A only. But class B can consume the member as if it is the owner. See, I can drive my father's car as if I am the owner, but still, the registration name is my father. Similarly, class B can call the methods as the method is its own, but internally, the methods belong to Class A.



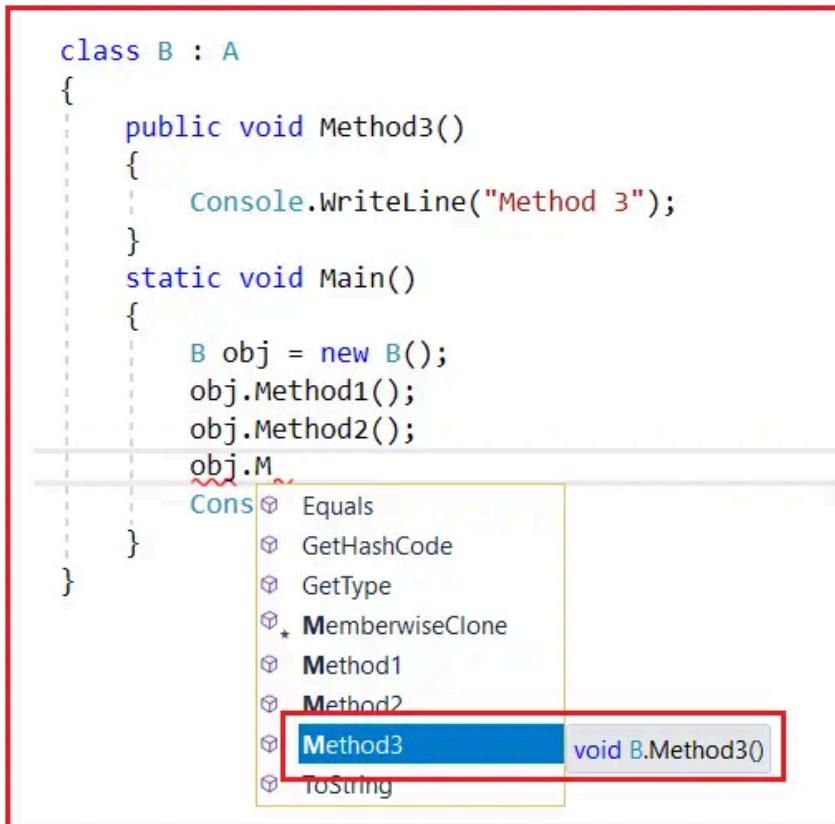
The complete code example is given below. In the below example, class A defined two members, and class B is inherited from Class A. In class B, within the Main method, we created an instance of class B and called the two methods.

```
using System;
namespace InheritanceDemo
{
    class A
    {
        public void Method1()
        {
            Console.WriteLine("Method 1");
        }
        public void Method2()
        {
            Console.WriteLine("Method 2");
        }
    }
    class B : A
    {
        static void Main()
        {
            B obj = new B();
            obj.Method1();
            obj.Method2();
            Console.ReadKey();
        }
    }
}
```

Output:

```
Method 1
Method 2
```

Now, let us add a new method, i.e., Method3 in Class B, as follows. Inside the Main method, if you see the method description, it shows that the method belongs to class B.



The complete example is given below.

```
using System;
namespace InheritanceDemo
{
    class A
    {
        public void Method1()
        {
            Console.WriteLine("Method 1");
        }
        public void Method2()
        {
            Console.WriteLine("Method 2");
        }
    }
    class B : A
    {
        public void Method3()
        {
            Console.WriteLine("Method 3");
        }
        static void Main()
        {

```

```

        B obj = new B();
        obj.Method1();
        obj.Method2();
        obj.Method3();
        Console.ReadKey();
    }
}

```

Output:

```

Method 1
Method 2
Method 3

```

How many methods are there in class B?

Now, you may have one question: how many methods are there in class B? The answer is 3. See, all the properties that my father has given to me, plus all the properties that I am earning, are my property only. So, what is my property means I don't say what I earned; I also say what my father has given to me. So, in the same way, how many methods are there in class B means 3 methods. Two methods were inherited from the parent class A plus one method which we defined in class B. So, we can say Class A contains two methods, and Class B contains 3 methods.

This is the simple process of Inheritance in C#. Simply put a colon (:) between the Parent and Child class. But when you are working with Inheritance, 6 things or rules are required to learn and remember. Let us learn those 6 important Rules one by one.

Rule1:

In C#, the parent classes constructor must be accessible to the child class; otherwise, the inheritance would not be possible because when we create the child class object, first it goes and calls the parent class constructor so that the parent class variable will be initialized and we can consume them under the child class.

Right now, in our example, both class A and class B have implicit constructors. Yes, every class in C# contains an implicit constructor if as a developer we did not define any constructor explicitly. We already learned it in our constructor section.

If a constructor is defined implicitly, then it is a public constructor. In our example, class B can access the class A implicit constructor as it is public. Now, let us define one explicit constructor in Class A as follows.

```

class A
{
    public A()
    {
        Console.WriteLine("Class A Constructor is Called");
    }
    public void Method1()

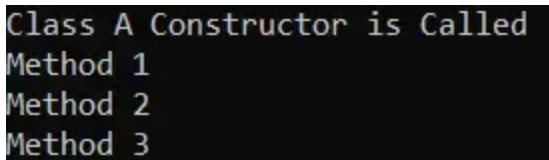
```

```

{
    Console.WriteLine("Method 1");
}
public void Method2()
{
    Console.WriteLine("Method 2");
}
}

```

With the above changes in place, if you run the application code, you will get the following output.



```

Class A Constructor is Called
Method 1
Method 2
Method 3

```

When you execute the code, the class A constructor is first called, and that is what you can see in the output. Why? This is because whenever the child class instance is created, the child class constructor will implicitly call its parent class constructors. This is a rule.

Right now, the child class contains an implicit constructor, and that implicit constructor calls the parent class constructor. But the Parent class A constructor is not implicitly. It is explicitly now, and inside that parent class constructor, we have written print statement and print statement printing some message on the console window.

But remember, if you are defining an explicit constructor, if you make that constructor private, and if you don't provide an access specifier, then by default, the class member's access specifier is private in C#. For example, modify class A as follows. As you can see, we have removed the access specifier from the constructor which makes it private.

```

class A
{
    A()
    {
        Console.WriteLine("Class A Constructor is Called");
    }
    public void Method1()
    {
        Console.WriteLine("Method 1");
    }
    public void Method2()
    {
        Console.WriteLine("Method 2");
    }
}

```

As you can see in the code, the Class A constructor is private, so it is not accessible to Class B. Now, if you try to run the code, you will get the following compile-time error as shown in the below image which tells **Class A Constructor is inaccessible due to its protection level.**

	Code	Description
	CS0122	'A.A()' is inaccessible due to its protection level

We are getting the above error because when we create an instance of the child class, the child class constructor will implicitly call its parent class constructors. Right now, the Class B constructor trying to call the Class A constructor, which is not accessible because that constructor is private.

Let us do one more thing. Let us define a constructor in Class B as follows. Let us make the class A constructor public; otherwise, the inheritance would not be possible.

```
using System;
namespace InheritanceDemo
{
    class A
    {
        public A()
        {
            Console.WriteLine("Class A Constructor is Called");
        }
        public void Method1()
        {
            Console.WriteLine("Method 1");
        }
        public void Method2()
        {
            Console.WriteLine("Method 2");
        }
    }
    class B : A
    {
        B()
        {
            Console.WriteLine("Class B Constructor is Called");
        }
        public void Method3()
        {
            Console.WriteLine("Method 3");
        }
        static void Main()
        {
            B obj = new B();
            obj.Method1();
            obj.Method2();
            obj.Method3();
            Console.ReadKey();
        }
    }
}
```

Output:

```
Class A Constructor is Called
Class B Constructor is Called
Method 1
Method 2
Method 3
```

As you can see in the above output, first Class A constructor is called, and then the Class B constructor is called. So, the point that you need to remember is execution always starts from the Parent class constructor. Why? When we create an instance of a child class, the child class constructor will implicitly call the parent class constructor. If that Parent class has a Parent class, then that Parent class constructor will call its Parent class constructor, and so on. Suppose you have 5 classes in inheritance, and if you are creating an instance of the 5th class, then the 5th class constructor will call the 4th class constructor, and 4th class constructor will call the 3rd class constructor and the 3rd class constructor will call the 2nd class constructor and 2nd class constructor will call the 1st class constructor. So, the execution, in this case, will start from the class 1 constructor, then the class 2 constructor, and the last constructor, in this case, will be the 5th class constructor.

Why is Class B Constructor not Public?

Here, you may have one question: the Class B constructor is not public. Why? See, the class B constructor does not need to be public because the class A constructor should be accessible to B, not Class B to class A. When the class B constructor is to be public, if Class B has a child class, then the class B constructor must be public. If class B does not have a child class, then there is no point in declaring the constructor as public. If you want, then you can also declare the constructor as public. In this case, that's not a matter at all.

So, always child class constructor implicitly calls the Parent class constructor, and hence, the parent class constructor should be accessible to the child class; otherwise, inheritance would not be possible. Now, you may have one question: why is the parent class constructor accessible to the child class?

Why is Parent Class Constructor Accessible to Child Class?

The reason is when the Parent class constructor is called, only the parent class members will be initialized, and then only they can be consumed under the child class. If the parent class members are not initialized, then you cannot consume them under the child class. If you want to consume them in the child class, they must be initialized. See, the Child Class is dependent on the Parent Class, so the Parent class must be initialized first, then only the consumption is possible in the Child class.

This is the first rule of Inheritance. Let us proceed and understand the second rule of Inheritance in C# with examples.

Rule2:

In inheritance, the child class can access the parent class members, but the parent classes can never access any members that are purely defined in the child class.

See, according to the law, children have rights to their parents's Property. However, the Parent does not have rights to the Children's property. It is only the children's responsibility to take care of their parents. But legally,

the Parent does not have rights to the child's property. Exactly in the same way, the Parent class can never access the child class members, which are purely defined in the child class.

Let us understand this with an example. Please have a look at the below code. Here, you can see inside the Main method, we are creating an instance of the Parent class, i.e., A, and trying to invoke the Parent class as well as child class methods. When we try to invoke Method3, which is purely defined in the child class, we will get a compile-time error.

```
using System;
namespace InheritanceDemo
{
    class A
    {
        public A()
        {
            Console.WriteLine("Class A Constructor is Called");
        }
        public void Method1()
        {
            Console.WriteLine("Method 1");
        }
        public void Method2()
        {
            Console.WriteLine("Method 2");
        }
    }
    class B : A
    {
        public B()
        {
            Console.WriteLine("Class B Constructor is Called");
        }
        public void Method3()
        {
            Console.WriteLine("Method 3");
        }
        static void Main()
        {
            A obj = new A();
            obj.Method1();
            obj.Method2();
            //The following line of code gives you compile time error
            obj.Method3();
            Console.ReadKey();
        }
    }
}
```

When you try to run the above code, you will get the following Compile Time Error.

Code	Description
CS1061	'A' does not contain a definition for 'Method3' and no accessible extension method 'Method3' accepting a first argument of type 'A' could be found (are you missing a using directive or an assembly reference?)

It is complaining that class 'A' does not contain a definition for 'Method3' and no accessible extension method 'Method3' accepting a first argument of type 'A' could be found (are you missing a using directive or an assembly reference?) and this makes sense.

So, this is the second rule of inheritance that a Parent class can never access any members of the child class who are purely defined in the child class. In this case, Method3 is purely defined in the child class B, and hence, we can not access this method using the Parent class object.

Rule3:

We can Initialize a Parent class variable by using the child class instance to make it a reference variable so that the reference will be consuming the memory of the child class instance. But in this case, also, we cannot call any pure child class members using the reference.

Now, you may have one question: **what is a reference?** The answer is a Reference is a pointer to an instance that does not have any memory allocation.

Let us understand this with an example. Please have a look at the below code. Inside the Main method, first, we create a variable p of class A, and here, p is a reference variable. It's not an instance, and it's a variable, i.e., an uninitialized copy

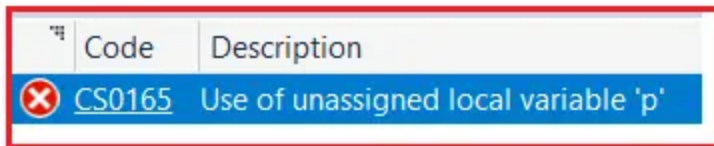
```
using System;
namespace InheritanceDemo
{
    class A
    {
        public A()
        {
            Console.WriteLine("Class A Constructor is Called");
        }
        public void Method1()
        {
            Console.WriteLine("Method 1");
        }
        public void Method2()
        {
            Console.WriteLine("Method 2");
        }
    }
    class B : A
    {
        public B()
        {
            Console.WriteLine("Class B Constructor is Called");
        }
    }
}
```

```

    public void Method3()
    {
        Console.WriteLine("Method 3");
    }
    static void Main()
    {
        A p; //p is a variable of class A
        p.Method1();
        p.Method2();
        Console.ReadKey();
    }
}

```

Now, if you try to run the above code, you will get the following Compile Time Error, i.e., **Use of unassigned local variable 'p'**.



This makes sense. The variable p is unassigned, and hence, we can not call any methods. It is not initialized. How to Initialize a reference variable? Initialization can be done by using the new keyword in C#. Let us see this. In the below example, we have initialized the parent class reference variable p using the child class instance and then called the parent class members. In the below example, the Main method code is self-explained, so please go through the comment lines.

```

using System;
namespace InheritanceDemo
{
    class A
    {
        public A()
        {
            Console.WriteLine("Class A Constructor is Called");
        }
        public void Method1()
        {
            Console.WriteLine("Method 1");
        }
        public void Method2()
        {
            Console.WriteLine("Method 2");
        }
    }
    class B : A
    {
        public B()
        {
            Console.WriteLine("Class B Constructor is Called");
        }
    }
}

```

```

    }
    public void Method3()
    {
        Console.WriteLine("Method 3");
    }
    static void Main()
    {
        A p; //p is a variable of class A
        B q = new B(); //q is an instance of Class B
        //We can initialize a Parent class variable using child class inst
follows
instance
        p = q; //now, p is a reference of parent class created by using cl
        //Now you can call members of A class as follows
        p.Method1();
        p.Method2();
        //We cannot call any pure child class members using the reference
        //p.Method3();
        Console.ReadKey();
    }
}

```

Output:

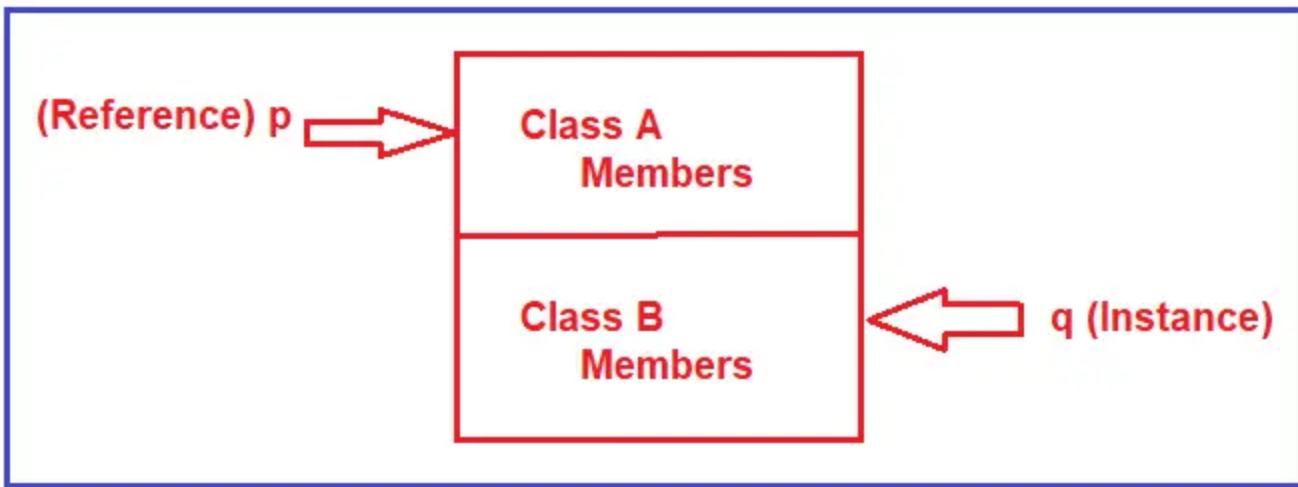
```

Class A Constructor is Called
Class B Constructor is Called
Method 1
Method 2

```

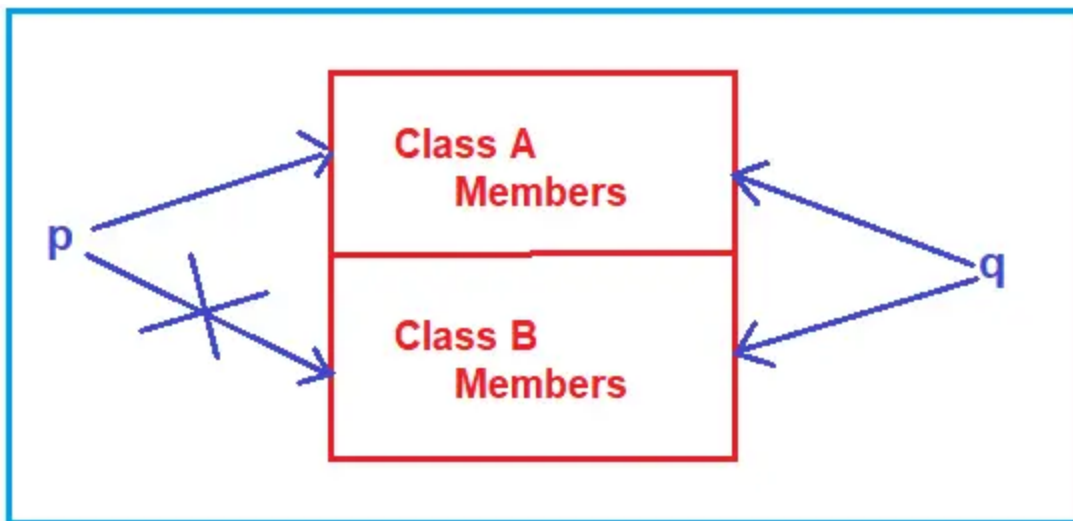
So, what is a Reference in C#?

References of a class will not have any memory allocation. They will be consuming the memory of the instance that is assigned to initialize them. For a better understanding, please have a look at the below diagram. Here, whenever we create an instance, memory is allocated for q. This instance will contain information about both Parent A and Child B classes. And p is a reference. And p is consuming the memory of q.



Note: The point that you need to remember is memory allocation is done for instances, not for references in C#. References are just pointers to instances.

Now, if observed, both p and q are accessing the same memory. But the point to understand is that even though p and q are accessing the same memory, using p, I cannot call any child class members. See the following diagram. As you can see in the diagram below, using p, we cannot call class B members, but using q, we can call both A and B class members.



Rule4:

Every class that is defined by us or predefined in the libraries of the language has a default parent class, i.e., the Object class of the System namespace, so the members (Equals, GetHashCode, GetType, and ToString) of the Object class are accessible from anywhere.

Generally, when we define a class, we think that we did not inherit it from any class. But by default, our class is Inherited from the Object class. So, Object is the parent class for all the classes defined in our Base Class Library as well as all the classes that we defined in our application.

Because Object is the parent class, four important methods (Equals, GetHashCode, GetType, and ToString) of the Object class can be called or accessed from anywhere. For a better understanding, please have a look at the below image. Here, we have created an instance of Object class, and when we say obj., the intelligence shows the four methods.

```
static void Main()
{
    Object obj = new Object();
    obj.|
    Cons
}
```

- Equals
- GetHashCode
- GetType
- ToString

bool object.Equals(object obj)
Determines whether the specified object is equal to the current object.
Note: Tab twice to insert the 'Equals' snippet.

Remember the above four methods can be accessed from everywhere. Every class can contain the Equals, GetHashCode, GetType, and ToString methods, and this is possible because every class in the .NET framework is inherited from the Object class.

Now, let us create an object of class A, and when you type obj., then the intelligence shows 6 methods, i.e., 2 methods (Method1 and Method2) from class A and four methods (Equals, GetHashCode, GetType, and ToString) from Object class shown in the below image.

```
static void Main()
{
    A obj = new A();
    obj.|
}
```

- Equals
- GetHashCode
- GetType
- Method1
- Method2
- ToString

bool object.Equals(object obj)
Determines whether the specified object is equal to the current object.
Note: Tab twice to insert the 'Equals' snippet.

Generally, when you compile your code, the compiler verifies if this class is inheriting from any other class. If yes, there are no issues. If no, automatically compiler makes this class inherited from the Object class. In our example, class A is not inherited from any class. At the time of compilation, this class will automatically inherit from the Object class.

On the other hand, when compiling class B, it will check whether class B is inherited from any other class. Yes, class B is inherited from class A. No, need to inherit from Object. The reason is that class A is already inheriting from Object. Because class A is already inherited from Object, for class B, Object is the Parent class, and maybe it is a grandparent.

So, the point that you need to remember is every class in the .NET Framework is either directly or indirectly inherited from the Object class.

Note: Object class supports all classes in the .NET Framework class hierarchy and provides low-level services to derived classes. This is the ultimate base class of all classes in the .NET Framework; it is the root of the type hierarchy.

What is the Default Parent Class in C#?

The Default Parent class is the Object class present in the System namespace.

Now, please have a look at the below example. Here, we have created three instances that are instances of the Object class, A class, and B class and called the GetType method. The GetType method returns the exact runtime type of the current instance. It will tell you the fully qualified name, i.e., the namespace and class name.

```
using System;
namespace InheritanceDemo
{
    class A
    {
        public A()
        {
            Console.WriteLine("Class A Constructor is Called");
        }
        public void Method1()
        {
            Console.WriteLine("Method 1");
        }
        public void Method2()
        {
            Console.WriteLine("Method 2");
        }
    }
    class B : A
    {
        public B()
        {
            Console.WriteLine("Class B Constructor is Called");
        }
        public void Method3()
        {
            Console.WriteLine("Method 3");
        }
        static void Main()
        {
            Object obj1 = new Object();
            Console.WriteLine($"obj1 type: {obj1.GetType()}");
            A obj2 = new A();
            Console.WriteLine($"obj2 type: {obj2.GetType()}");
            B obj3 = new B();
            Console.WriteLine($"obj3 type: {obj3.GetType()}");
            Console.ReadKey();
        }
    }
}
```

Output:

```
obj1 type: System.Object
Class A Constructor is Called
obj2 type: InheritanceDemo.A
Class A Constructor is Called
Class B Constructor is Called
obj3 type: InheritanceDemo.B
```

Constructor Execution in the above Example:

1. When we create an instance of the Object class, only the Object class constructor is called.
2. But when we create an instance of Class A, two constructors are called. First, it will execute the Object class constructor, and then it will execute the Class A constructor.
3. When we create an instance of Class B, then three constructors are executed. First, it will execute the Object class constructor; then it will execute the class A constructor; and last, it will execute the class B Constructor.

Rule5:

In C#, we don't have support for multiple inheritances through classes. What we are provided is only a Single Inheritance through classes. That means with classes, only one immediate parent class is allowed (i.e., Single, Multilevel, and Hierarchical supported), and more than one immediate parent class is not allowed in C# with classes (i.e., Multiple and Hybrid are not supported). In our next article, we will discuss this rule in detail.

Rule6:

In Rule1, we learned whenever the child class instance is created, the child class constructor will implicitly call its parent classes constructor, but if the parent classes constructor is parameterless. If the constructor of the Parent class is parameterized, then the Child class constructor cannot implicitly call its Parent's constructor. So, to overcome this problem, it is the responsibility of the programmer to explicitly call the Parent classes constructor from the child class constructor and pass values to those parameters. To call the Parent's constructor from the child class, we need to use the base keyword.

Let us understand this with an example. Let us make the Parent class constructor parameterized as follows. Here, the constructor takes an integer parameter and prints that value on the console window.

```
using System;
namespace InheritanceDemo
{
    class A
    {
        public A(int number)
        {
            Console.WriteLine($"Class A Constructor is Called : {number}");
        }
        public void Method1()
        {
```

```

        Console.WriteLine("Method 1");
    }
    public void Method2()
    {
        Console.WriteLine("Method 2");
    }
}
class B : A
{
    public B()
    {
        Console.WriteLine("Class B Constructor is Called");
    }
    public void Method3()
    {
        Console.WriteLine("Method 3");
    }
    static void Main()
    {
        B obj = new B();
        Console.ReadKey();
    }
}

```

Now, if you compile the code, then you will see that class B is giving one error, as shown in the below image.

Code	Description
 CS7036	There is no argument given that corresponds to the required formal parameter 'number' of 'A.A(int)'

It is complaining that “**There is no argument given that corresponds to the required formal parameter 'number' of 'A.A(int)'**” and this makes sense. This is because the class B constructor implicitly calls the class A constructor. But, if you want to call the class A constructor, it requires an integer parameter now. Without passing the parameter, we cannot call the class A constructor. So, now, the class B constructor is unable to call the class A constructor.

Why is it unable to call the constructor?

Earlier, the constructor was parameterless, so it directly calls the Parent class constructor. Right now, the constructor is parameterized. If you want to call it, you require a value now. The class B constructor does not know what value to pass the class A constructor. This is the reason why we are getting an error in class B, not in class A. Why? Because class B is not able to call the class A constructor.

So, what to do? Implicit calling is not working. So, to resolve the error, we need to go for explicit calling. How to call? See the following code. Here, the keyword `base` refers to the parent class, i.e., class A. Here we need to pass the value for the Base class or Parent class constructor.

Calling the Base class or Parent class i.e.
Class A constructor by passing the value 10



```
class B : A
{
    public B() : base(10)
    {
        Console.WriteLine("Class B Constructor is Called");
    }
    public void Method3()...
    static void Main()...
}
```

So, here we are passing the value 10 to the Parent class, i.e., A constructor. This value of 10 will be received by the parent class constructor. The complete code is given below.

```
using System;
namespace InheritanceDemo
{
    class A
    {
        public A(int number)
        {
            Console.WriteLine($"Class A Constructor is Called : {number}");
        }
        public void Method1()
        {
            Console.WriteLine("Method 1");
        }
        public void Method2()
        {
            Console.WriteLine("Method 2");
        }
    }
    class B : A
    {
        public B() : base(10)
        {
            Console.WriteLine("Class B Constructor is Called");
        }
        public void Method3()
        {
            Console.WriteLine("Method 3");
        }
        static void Main()
        {
            B obj = new B();
        }
    }
}
```

```

        Console.ReadKey();
    }
}

```

Output:

```

Class A Constructor is Called : 10
Class B Constructor is Called

```

How to pass dynamic value to Parent class constructor in C#?

In our previous example, we have hardcoded the value, i.e., 10. So, every time we create an instance, the same value will be assigned to the parent constructor. But if we want, then we can also pass dynamic value. Let us see this with an example. In the below example, the child class, i.e., class B constructor, takes one parameter and passes that parameter value to the parent class, i.e., Class A constructor. And when we are creating the instance of Class B, we need to pass the parameter value.

```

using System;
namespace InheritanceDemo
{
    class A
    {
        public A(int number)
        {
            Console.WriteLine($"Class A Constructor is Called : {number}");
        }
        public void Method1()
        {
            Console.WriteLine("Method 1");
        }
        public void Method2()
        {
            Console.WriteLine("Method 2");
        }
    }
    class B : A
    {
        public B(int num) : base(num)
        {
            Console.WriteLine("Class B Constructor is Called");
        }
        public void Method3()
        {
            Console.WriteLine("Method 3");
        }
        static void Main()
        {
            B obj1 = new B(10);
            B obj2 = new B(20);
        }
    }
}

```

```

    }
    {
        B obj3 = new B(30);
        Console.ReadKey();
    }
}

```

Output:

```

Class A Constructor is Called : 10
Class B Constructor is Called
Class A Constructor is Called : 20
Class B Constructor is Called
Class A Constructor is Called : 30
Class B Constructor is Called

```

So, in the above example, when we are creating the instance, we are passing the value. The value first reaches the child class constructor, and the child class constructor passes the same value to the parent class constructor. If you want, then you can also use the same value in the child class.

So, these are the six rules that you should remember while working with Inheritance in C#.

Advantages of Inheritance in C#:

Code reusability: We can reuse the members of the parent class or base class in the child class or derived class. So, there is no need to redefine the members again in the child class. So, less code is required in the class.

In the next article, I will discuss [Types of Inheritance in C#](#) with Examples. In this article, I try to explain **Inheritance in C#** with Examples, and I hope you enjoy this article. I would like to have your feedback. Please post your feedback, questions, or comments about this article.